Ultrasonic Sensor Code Explanation

07 May 2026 10:07

-> The goal of code is to measure distance using HC-SR04 ultrasonic sensor with Raspberry Pi GPIOs using libgpiod, libra and indicate distance using LEDs.
Note: Use resistors when connecting leds to gpio, also connect a voltage divider between echo pin and gpio pin.

-> Working of the sensor:
The HC-SR04 ultrasonic sensor measures distance using ultrasonic sound waves. The Raspberry Pi sends a 10µs HIGH pulse to the TRIG pin, which triggers the sensor to transmit an ultrasonic burst. The sound wave travels through air, reflects from an object, and returns to the sensor.

The ECHO pin remains HIGH for the total time taken by the sound wave to travel to the object and back. The Raspberry Pi measures this HIGH pulse duration and calculates the distance using the speed of sound.

-> In distance calculation, we convert m/s to cm/us because the timer we are using is in us(micro sec).

Speed of sound ≈ 343 m/s
343 m/s => 34300 cm/s => 34300 / 1000000 cm/µs ≈ 0.0343 cm/µs

Distance is calculated using: distance = speed × time

However, the measured time represents the total round-trip time of the ultrasonic wave (sensor → object → sensor). Therefore:

distance = (duration × 0.0343) / 2

Rearranging: distance ≈ duration / 58

Thus, dividing the echo pulse duration by 58 directly gives the distance in centimeters.

-> Program Flow

1. Initialize GPIO chip using libgpiod(refer previous page linb/func info)
2. Configure TRIG pin as output
3. Configure ECHO pin as input
4. Configure LED GPIOs as output
5. Send 10µs HIGH pulse to TRIG pin
6. Wait for ECHO pin to go HIGH
7. Start timer when ECHO becomes HIGH
8. Wait for ECHO pin to go LOW
9. Stop timer when ECHO becomes LOW
10. Calculate pulse duration
11. Convert duration into distance
12. Print measured distance
13. Control LEDs based on distance
14. Repeat continuously inside while loop

-> Drawbacks or Errors faced
The HC-SR04 ECHO pin outputs 5V logic, while Raspberry Pi GPIOs support only 3.3V. A voltage divider was required to safely interface the ECHO signal.
LED brightness was very low because only 4.7kΩ resistors were available. Higher resistance limited the LED current significantly.
The polling-based while loops can block the CPU indefinitely if the sensor fails to respond or wiring is incorrect.
Accurate timing measurement in Linux userspace is difficult because Linux is not a real-time operating system.
Small fluctuations were observed in distance readings due to environmental noise and timing variations.

-> Hardware Connections:

1. HC-SR04(Sensor) Connections
VCC:
HC-SR04 VCC → Raspberry Pi Pin 2 (5V)

GND
HC-SR04 GND → Raspberry Pi Pin 6 (GND)

TRIG pin
HC-SR04 TRIG→ Raspberry Pi GPIO23(Physical Pin 16)

2. ECHO Voltage Divider Connections
ECHO to resistor

HC-SR04 ECHO → one side of 4.7kΩ (1) resistor

GPIO24 connection
Other side of 4.7kΩ(1) resistor → Raspberry Pi GPIO24(Physical Pin 18)

Pull-down resistor
Same GPIO24 junction → one side of 10kΩ resistor(if available) -> Other side of 10kΩ resistor → GND (pin 9)
Gnd rail from raspberry pi, physical pin 14 to breadboard

3. Green LED Connections
Raspberry Pi GPIO17(Physical Pin 11) → resistor → Green LED anode (+ long leg)
Green LED cathode (- short leg) → Breadboard GND rail

4. Yellow LED Connections
Raspberry Pi GPIO27(Physical Pin 13) → resistor → Yellow LED anode (+ long leg)
Yellow LED cathode (- short leg) → Breadboard GND rail

Code

08 May 2026 12:14

```
#include <stdio.h>
#include <unistd.h>
#include <time.h>
#include <gpiod.h>

#define CHIP "/dev/gpiochip0"
#define TRIG 23
#define ECHO 24

#define LED_YELLOW 17
#define LED_GREEN 27
long get_time_us()
{
   struct timespec t;
   clock_gettime(CLOCK_MONOTONIC, &t);
   return t.tv_sec * 1000000 + t.tv_nsec / 1000;
}

int main()
{
   struct gpiod_chip *chip;
   struct gpiod_line_settings *trig_set, *echo_set, *led_set;
   struct gpiod_line_config *line_cfg;
   struct gpiod_request_config *req_cfg;
   struct gpiod_line_request *req;

   int trig = TRIG;
   int echo = ECHO;
   int led1=LED_GREEN;
   int led2=LED_YELLOW;

   chip = gpiod_chip_open(CHIP);

   trig_set = gpiod_line_settings_new();
   gpiod_line_settings_set_direction(trig_set, GPIOD_LINE_DIRECTION_OUTPUT);

   echo_set = gpiod_line_settings_new();
   gpiod_line_settings_set_direction(echo_set, GPIOD_LINE_DIRECTION_INPUT);

   led_set = gpiod_line_settings_new();
   gpiod_line_settings_set_direction(led_set, GPIOD_LINE_DIRECTION_OUTPUT);

   line_cfg = gpiod_line_config_new();

   gpiod_line_config_add_line_settings(line_cfg, &trig, 1, trig_set);
   gpiod_line_config_add_line_settings(line_cfg, &echo, 1, echo_set);
   gpiod_line_config_add_line_settings(line_cfg, &led1, 1, led_set);
   gpiod_line_config_add_line_settings(line_cfg, &led2, 1, led_set);

   req_cfg = gpiod_request_config_new();
   gpiod_request_config_set_consumer(req_cfg, "ultrasonic");

   req = gpiod_chip_request_lines(chip, req_cfg, line_cfg);
```

```
    long start, end;
    float distance;

    while (1)
    {
        // trigger pulse
        gpiod_line_request_set_value(req, TRIG, 1);
        usleep(10);
        gpiod_line_request_set_value(req, TRIG, 0);

        // wait echo HIGH
        while (gpiod_line_request_get_value(req, ECHO) == 0);

        start = get_time_us();

        // wait echo LOW
        while (gpiod_line_request_get_value(req, ECHO) == 1);

        end = get_time_us();

        distance = (end - start) / 58.0;

        printf("Distance: %.2f cm\n", distance);

        //keep both the leds off
        gpiod_line_request_set_value(req, LED_GREEN, 0);
        gpiod_line_request_set_value(req, LED_YELLOW, 0);

        if(distance < 15)
        {
            printf("Object very close\n");
            gpiod_line_request_set_value(req, LED_GREEN, 1);
        }
        else if (distance<30)
        {
            printf("Object nearby\n");
            gpiod_line_request_set_value(req, LED_YELLOW, 1);

        }
        else
        {
            printf("Object not in close range\n");
        gpiod_line_request_set_value(req, LED_GREEN, 0);
        gpiod_line_request_set_value(req, LED_YELLOW, 0);
        }

        usleep(60000);
    }
}
```